Program-2

Feed Forward Neural Network for IMDB Movie Review Classification

Aim: Implement multi-layer perceptron algorithm for Binary Classification – IMDB Movie reviews.

Introduction to IMDB Dataset:

The **IMDb dataset** is to text processing. It is the standard dataset for **Natural Language Processing (NLP)**. It consists of movie reviews from the Internet Movie Database (IMDb), labeled by sentiment (positive or negative). The goal is **Sentiment Analysis** (Binary Classification).

- **Input:** A text review written by a human (e.g., *"This movie was good but confusive!"*).
- **Output:** A classification of either **Positive** or **Negative**.
- **Size**: The version in Keras/Tensorflow contains **50,000 highly polarized reviews**.
    - **Training Set**: 25,000 reviews for training a model.
    - **Test Set**: 25,000 reviews for evaluating the trained model.
    - **Balance:** It is perfectly balanced—50% of the reviews are positive, and 50% are negative.
- **Labels**: Neural networks cannot understand text; they only understand numbers. Keras pre-processes the data:
    - It creates a dictionary of all words used in the reviews.
    - It ranks them by frequency (how often they appear).
    - It replaces every word with its rank.
- **Example:**
    - **3** might represent the 3rd most common word (like "the").
    - **88** might represent the 88th most common word (like "great").
    - So, the sentence *"The movie was great"* might look like this to the computer: [3, 15, 12, 88]

Sentiment Analysis:

- Sentiment Analysis (also known as Opinion Mining) is the process of using computers to determine the emotional tone behind words.
- While humans can easily tell if a sentence is sarcastic, angry, or happy, computers generally only see characters and numbers. Sentiment analysis bridges this gap by classifying text into categories, most commonly:
    - Positive ("I loved this movie!")
    - Negative ("The service was terrible.")
    - Neutral ("The package arrived at 5 PM.")

Modeling the network:

- Layer-1: Embedding layer: It looks up a vector for each word. Instead of a single number, "Happy" becomes a list of 16 floating-point numbers that represent its *meaning*. The model learns these meanings during training.
- Layer-2: GlobalAveragePooling1D: This layer calculates the average of all word vectors in the review. It condenses the sequence into a single "gist" vector.
- Layer-3: Standard hidden layer to interpret the "gist" vector. It contains 16 neurons with activation function – 'relu'.
- Layer-4: Output layer: It gives the output containing one neuron with sigmoid activation function.

  Output Closer to 0: Negative Sentiment.

  Output Closer to 1: Positive Sentiment.

Modules used:

1. Tensorflow, Keras : for modeling the network
2. Matplotlib : for graphical visualization of result

Program:

```
#Import the required libraries
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.datasets import imdb
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.sequence import pad_sequences
import matplotlib.pyplot as plt
#Load the dataset and Perform Padding
num_words=10000
(x_train,y_train), (x_test,y_test)=imdb.load_data(num_words=num_words)
maxlen=200
x_train=pad_sequences(x_train, maxlen=maxlen)
x_test=pad_sequences(x_test, maxlen=maxlen)
#Model the network (bag of words style)
model=keras.Sequential([
   layers.Embedding(num_words, 32, input_length=maxlen),
   layers.GlobalAveragePooling1D(),
   layers.Dense(16,activation='relu'),
   layers.Dense(1,activation='sigmoid')
])
```

```
#Define the metrics
model.compile(optimizer='adam', loss='binary_crossentropy',metrics=['accuracy'])
#Train and Evaluate the model
history=model.fit(x_train, y_train, epochs = 10, batch_size=120, validation_split=0.2)
test_loss, test_acc=model.evaluate(x_test,y_test)
print(f"Test accuracy:{test_acc:.4f}")
print(f"Test loss:{test_loss:.4f}")
#Plot the graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.savefig('Model Accuracy.tiff')
plt.show()
```

Output: Test accuracy: 0.8673

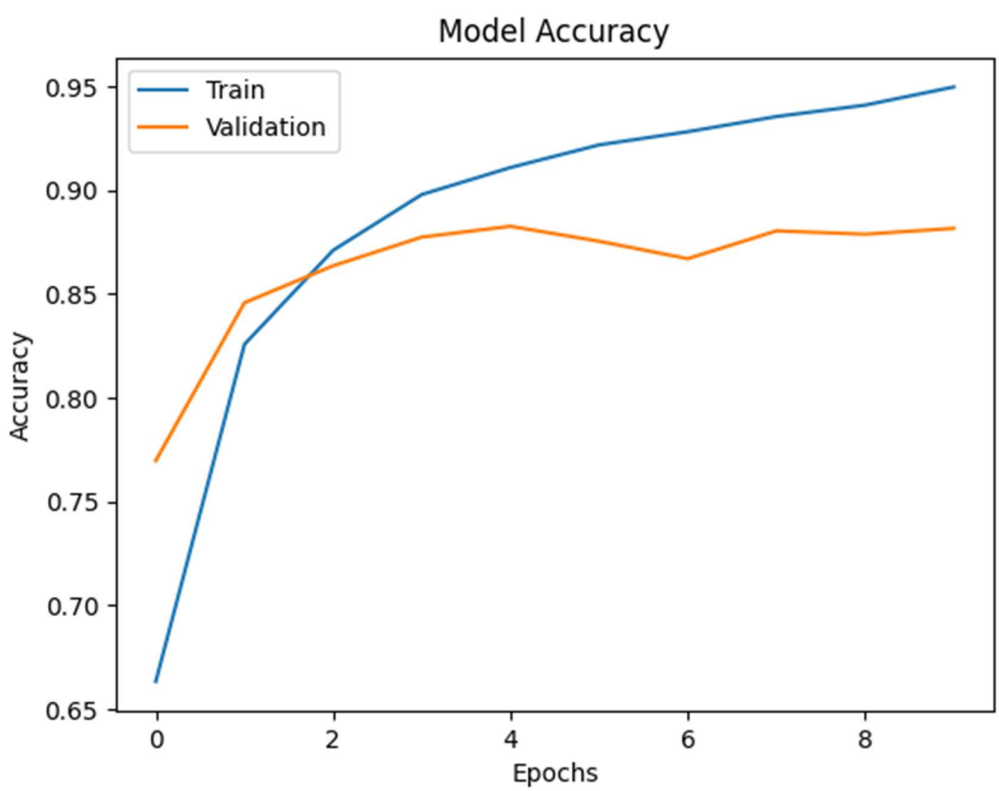

Fig. 1 Model Accuracy plot